

Cambridge International AS & A Level

COMPUTER SCIENCE**9618/32**

Paper 3 Advanced Theory

May/June 2021**MARK SCHEME**Maximum Mark: 75

Published

This mark scheme is published as an aid to teachers and candidates, to indicate the requirements of the examination. It shows the basis on which Examiners were instructed to award marks. It does not indicate the details of the discussions that took place at an Examiners' meeting before marking began, which would have considered the acceptability of alternative answers.

Mark schemes should be read in conjunction with the question paper and the Principal Examiner Report for Teachers.

Cambridge International will not enter into discussions about these mark schemes.

Cambridge International is publishing the mark schemes for the May/June 2021 series for most Cambridge IGCSE™, Cambridge International A and AS Level components and some Cambridge O Level components.

This document consists of **10** printed pages.

Generic Marking Principles

These general marking principles must be applied by all examiners when marking candidate answers. They should be applied alongside the specific content of the mark scheme or generic level descriptors for a question. Each question paper and mark scheme will also comply with these marking principles.

GENERIC MARKING PRINCIPLE 1:

Marks must be awarded in line with:

- the specific content of the mark scheme or the generic level descriptors for the question
- the specific skills defined in the mark scheme or in the generic level descriptors for the question
- the standard of response required by a candidate as exemplified by the standardisation scripts.

GENERIC MARKING PRINCIPLE 2:

Marks awarded are always **whole marks** (not half marks, or other fractions).

GENERIC MARKING PRINCIPLE 3:

Marks must be awarded **positively**:

- marks are awarded for correct/valid answers, as defined in the mark scheme. However, credit is given for valid answers which go beyond the scope of the syllabus and mark scheme, referring to your Team Leader as appropriate
- marks are awarded when candidates clearly demonstrate what they know and can do
- marks are not deducted for errors
- marks are not deducted for omissions
- answers should only be judged on the quality of spelling, punctuation and grammar when these features are specifically assessed by the question as indicated by the mark scheme. The meaning, however, should be unambiguous.

GENERIC MARKING PRINCIPLE 4:

Rules must be applied consistently, e.g. in situations where candidates have not followed instructions or in the application of generic level descriptors.

GENERIC MARKING PRINCIPLE 5:

Marks should be awarded using the full range of marks defined in the mark scheme for the question (however; the use of the full mark range may be limited according to the quality of the candidate responses seen).

GENERIC MARKING PRINCIPLE 6:

Marks awarded are based solely on the requirements as defined in the mark scheme. Marks should not be awarded with grade thresholds or grade descriptors in mind.

Question	Answer	Marks																
1(a)	Working: one mark for calculation of the mantissa and one mark for calculation or use of the exponent Exponent: one from: = 0.11101×2^3 // 0.11101×2^{11} // 0.11101×10^3 // 0.11101×10^{11} = 1.00011×2^3 // 1.00011×2^{11} // 1.00011×10^3 // 1.00011×10^{11} = appropriate shifting of binary point for +7.25 Mantissa: one from: = 111.01 (conversion to binary +7.25 – 10 bits) = 0111010000 (mantissa 10 bits for +7.25 = 1000101111(one's complement mantissa for –7.25) = 1000110000 (two's complement mantissa for –7.25) Correct Answer (Max 1) Mantissa <table><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table> Exponent <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	1	0	0	0	1	1	0	0	0	0	0	0	0	0	1	1	3
1	0	0	0	1	1	0	0	0	0									
0	0	0	0	1	1													
1(b)	One mark for working out the exponent One mark for working out the mantissa One mark for the correct answer Example answers • =1.011000111×2^7 (exponent is 7) • =10110001.11 // $-128 + 32 + 16 + 1 + 0.5 + 0.25$ // convert to positive 01001110.01 (and add a minus sign to the answer) • -78.25	3																
1(c)	One mark for working One mark for correct mantissa One mark for correct exponent Example answers Number of places added to exponent for normalisation –6 for number to retain its value // mantissa moved 6 places left Mantissa <table><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table> Exponent <table><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	0	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	3
0	1	1	1	0	0	0	0	0	0									
1	0	0	0	0	1													
1(d)(i)	One mark for each correct marking point (Max 3) • Requires 11 bits / more than 10 bits to store (accurately) / reference to maximum (positive) number that can be stored = 511 • Denary 513 in binary is 1000000001 // Normalised: 0.1000000001 • Results in overflow	3																

Question	Answer	Marks
1(d)(ii)	One mark for each correct marking point (Max 2) - The number of bits for the mantissa must be increased 11/12 bits mantissa and 5/4 bits exponent	2

Question	Answer	Marks
2(a)	One mark for each correct marking point (Max 2) - To create a new data type (from existing data types) - To allow data types not available in a programming language to be constructed // To extend the flexibility of the programming language	2
2(b)(i)	TYPE SchoolDay = (Monday, Tuesday, Wednesday, Thursday, Friday)	1
2(b)(ii)	TYPE WeekEnd = (Saturday, Sunday)	1
2(c)	One mark for each marking point (Max 4) - TYPE ClubMeet and ENDTYPE correct - DECLARE FirstName and DECLARE LastName included with correct data types - DECLARE Schoolday included with correct data types from part 2(b)(i) - DECLARE Weekend included with correct data types from part 2(b)(ii) Example answer <pre> TYPE ClubMeet DECLARE FirstName : STRING DECLARE LastName : STRING DECLARE Schoolday : SchoolDay DECLARE Weekend : WeekEnd ENDTYPE </pre>	4

Question	Answer	Marks														
3(a)	One mark for each correct line from Operating System Term to Description <table><thead><tr><th>OS term</th><th>Description</th></tr></thead><tbody><tr><td>Multi-tasking</td><td>Using secondary storage to simulate additional main memory</td></tr><tr><td>Paging</td><td>Managing the processes running on the CPU</td></tr><tr><td>Interrupt handling</td><td>Managing the execution of many programs that appear to run at the same time</td></tr><tr><td>Scheduling</td><td>Locating non-contiguous blocks of data and relocating them</td></tr><tr><td>Virtual memory</td><td>Transferring control to another routine when a service is required</td></tr><tr><td></td><td>Reading/writing same-size blocks of data from/to secondary storage when required</td></tr></tbody></table>	OS term	Description	Multi-tasking	Using secondary storage to simulate additional main memory	Paging	Managing the processes running on the CPU	Interrupt handling	Managing the execution of many programs that appear to run at the same time	Scheduling	Locating non-contiguous blocks of data and relocating them	Virtual memory	Transferring control to another routine when a service is required		Reading/writing same-size blocks of data from/to secondary storage when required	5
OS term	Description															
Multi-tasking	Using secondary storage to simulate additional main memory															
Paging	Managing the processes running on the CPU															
Interrupt handling	Managing the execution of many programs that appear to run at the same time															
Scheduling	Locating non-contiguous blocks of data and relocating them															
Virtual memory	Transferring control to another routine when a service is required															
	Reading/writing same-size blocks of data from/to secondary storage when required															
3(b)	One mark for each correct statement (Max 4) • An interpreter examines source code one statement at a time • Check each statement for errors • ...If no error is found the statement is executed • ...If an error is found this is reported and the interpreter halts • Interpretation is repeated for every iteration in repeated sections of code/in loops • Interpretation has to be repeated every time the program is run	4														

Question	Answer	Marks
4(a)(i)	One mark for each correct marking point (Max 2) • Reverse Polish Notation provides an unambiguous method of representing an expression • ... reading from left to right • ...without the need to use brackets • ...with no need for rules of precedence / BODMAS	2

Question	Answer	Marks
4(a)(ii)	One mark for identification of the data structure, One mark for a sensible reason Either: Structure: stack The operands are popped from the stack in the reverse order to how they were pushed Or: Structure: Binary tree A (binary) tree allows both infix and postfix to be evaluated (tree traversal)	2
4(b)	$a \ b - a \ c + * \ 7 \ /$	1
4(c)	$a \ / \ b * \ 4 - (a + b)$	1
4(d)	1 mark for correct structure 1 mark for correct substitution $(a + b) \ / \ (c \ / \ d)$ $(17 + 3) \ / \ (48 \ / \ 12)$	2

Question	Answer	Marks												
5(a)	Working (Max 3) May be seen on diagram • Initialisation: setting Base to 0 • ... and the rest of the towns to ∞ • Evidence to show values at nodes being updated • Evidence to show 'visited node(s)' May be seen in working section of paper • Evidence to show calculation of at least one route • Evidence to show more than one route has been calculated for at least one town Correct Answer (Max 2) One mark for four correct values... ... One mark for all values correct <table><tr><th>Town 1</th><th>Town 2</th><th>Town 3</th><th>Town 4</th><th>Town 5</th><th>Town 6</th></tr><tr><td>3</td><td>5</td><td>2</td><td>9</td><td>3</td><td>8</td></tr></table>	Town 1	Town 2	Town 3	Town 4	Town 5	Town 6	3	5	2	9	3	8	5
Town 1	Town 2	Town 3	Town 4	Town 5	Town 6									
3	5	2	9	3	8									

Question	Answer	Marks
5(b)	One mark for each correct marking point (Max 3) • Artificial Neural Networks can be represented using graphs • Graphs provide structures for relationships // graphs provide relationships between nodes • AI problems can be defined/solved as finding a path in a graph • Graphs may be analysed/ingested by a range of algorithms • ...e.g. A* / Dijkstra's algorithm • ...used in machine learning. • Example of method e.g. Back propagation of errors / regression methods	3

Question	Answer	Marks
6	One mark for each correct benefit (Max 2) • Accuracy – Ensures accurate delivery of the message • Completeness – Missing packets can be easily detected and a re-send request sent so the message arrives complete • Resilience – if a network changes the router can detect this and send the data another way to ensure it arrives • Path also available to other users // Doesn't use whole bandwidth // allows simultaneous use of channel by multiple users • Better security as packets hashed and sent by different routes. One mark for each correct drawback (Max 2) • Time delays to correct errors // Network problems may introduce errors in packets • Requires complex protocols for delivery • Unsuitable for real time transmission applications	4

Question	Answer	Marks																																																																								
7(a)	One mark for working, (all three columns P, Q and R) One mark for each correct column Y, Z <table><tr><th>A</th><th>B</th><th>C</th><th>P</th><th>Q</th><th>R</th><th>Y</th><th>Z</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td></tr></table>	A	B	C	P	Q	R	Y	Z	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	1	0	1	0	0	1	0	0	1	1	1	0	1	0	1	1	0	0	1	0	0	1	0	1	0	1	1	0	1	0	1	1	1	0	0	1	0	0	1	1	1	1	0	1	0	1	1	3
A	B	C	P	Q	R	Y	Z																																																																			
0	0	0	0	0	0	0	0																																																																			
0	0	1	0	0	0	1	0																																																																			
0	1	0	1	0	0	1	0																																																																			
0	1	1	1	0	1	0	1																																																																			
1	0	0	1	0	0	1	0																																																																			
1	0	1	1	0	1	0	1																																																																			
1	1	0	0	1	0	0	1																																																																			
1	1	1	0	1	0	1	1																																																																			
7(b)	Full adder	1																																																																								
7(c)	One mark for each point $Y = \bar{A} \bar{B} C + \bar{A} B \bar{C} + A \bar{B} \bar{C} + A B C$ Purpose: Sum bit $Z = \bar{A} B C + A \bar{B} C + A B \bar{C} + A B C$ Purpose: Carry output	4																																																																								

Question	Answer	Marks
8(a)	One mark for each correct marking point (Max 2) • The initial order of the data • The number of data items to be sorted • The efficiency of the sorting algorithm	2

Question	Answer	Marks
8(b)	One mark for each marking point (max 6) MP1 Use of FOR loop to cycle through the <u>whole year group</u> MP2 Temporary storage of the score being 'inserted' MP3 Temporary storage of the corresponding name elements MP4 Use of WHILE loop with correct exit clause MP5 Moving of all three elements of data to next array elements MP6 Correct updating of counter variable MP7 Final insertion of all three data elements Example algorithm <pre> YearSize ← 249 FOR Student ← 2 to YearSize Temp1 ← Score[Student] Temp2 ← Name[Student,1] Temp3 ← Name[Student,2] Counter ← Student WHILE Counter > 1 AND Score[Counter - 1] < Temp1 Score[Counter] ← Score[Counter - 1] Name[Counter,1] ← Name[Counter - 1,1] Name[Counter,2] ← Name[Counter - 1,2] Counter ← Counter - 1 ENDWHILE Score[Counter] ← Temp1 Name[Counter,1] ← Temp2 Name[Counter,2] ← Temp3 NEXT Student </pre>	6

Question	Answer	Marks
9(a)	One mark for each correct marking point (Max 2) • Imperative languages use variables • ... which are changed using (assignment) statements • ... they rely on a method of repetition / iteration. • The statements provide a sequence of commands for the computer to perform • ... in the order written / given • ... each line of code changes something in the program run.	2
9(b)	One mark for each correct marking point (Max 2) • Instructs a program on what needs to be done instead of how to do it • ... using facts and rules • ... using queries to satisfy goals. • Can be logical or functional • Logical - states a program as a set of logical relations • Functional – constructed by applying functions to arguments / uses a mathematical style	2

Question	Answer	Marks										
9(c)	One mark for each correct programming paradigm (Max 4) <table><tr><th>Program code example</th><th>Programming paradigm</th></tr><tr><td><pre>male(john). female(ethel). parent(john, ethel).</pre></td><td>Declarative</td></tr><tr><td><pre>FOR Counter = 1 TO 20 X = X * Counter NEXT Counter</pre></td><td>Procedural / imperative</td></tr><tr><td><pre>Start: LDD Counter INC ACC STO Counter</pre></td><td>Low-level / assembly</td></tr><tr><td><pre>public class Vehicle { private speed; public Vehicle() { speed = 0; } }</pre></td><td>Object oriented / (OOP)</td></tr></table>	Program code example	Programming paradigm	<pre>male(john). female(ethel). parent(john, ethel).</pre>	Declarative	<pre>FOR Counter = 1 TO 20 X = X * Counter NEXT Counter</pre>	Procedural / imperative	<pre>Start: LDD Counter INC ACC STO Counter</pre>	Low-level / assembly	<pre>public class Vehicle { private speed; public Vehicle() { speed = 0; } }</pre>	Object oriented / (OOP)	4
Program code example	Programming paradigm											
<pre>male(john). female(ethel). parent(john, ethel).</pre>	Declarative											
<pre>FOR Counter = 1 TO 20 X = X * Counter NEXT Counter</pre>	Procedural / imperative											
<pre>Start: LDD Counter INC ACC STO Counter</pre>	Low-level / assembly											
<pre>public class Vehicle { private speed; public Vehicle() { speed = 0; } }</pre>	Object oriented / (OOP)											